

Ex.No: 14

PROGRAM

```
# (Save the notebook as Naivebayes_classifier.ipynb)
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
# Load Play tennis dataset (built into scikit-learn, no upload needed)
data = pd.read_csv("play_tennis.csv")
print("First five records:")
display(data.head())
print("Dataset Shape:", data.shape)print("\nClass distribution:")
print(data["Species"].value_counts())
```

Output:

First five records:

	Outlook	Temperature	Humidity	Wind	Play
0	Sunny	Hot	High	Weak	No
1	Sunny	Hot	High	Strong	No
2	Overcast	Hot	High	Weak	Yes
3	Rain	Mild	High	Weak	Yes
4	Rain	Cool	Normal	Weak	Yes

Dataset Shape: (14, 5)

```
# Encode each categorical column
encoded_data = data.copy()
encoders = {}
for column in encoded_data.columns:
    encoders[column] = LabelEncoder()
    encoded_data[column] = encoders[column].fit_transform(encoded_data[column])
X = encoded_data.drop("Play", axis=1)
y = encoded_data["Play"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)
model = GaussianNB()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Actual Values  :", y_test.to_numpy())
print("Predicted Values:", y_pred)
print("Accuracy       :", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

```
print("\nClassification Report:")
print(classification_report(y_test, y_pred, zero_division=0))
```

Output:

```
Actual Values   : [1 1 0 1 0]
Predicted Values: [1 0 0 1 1]
Accuracy        : 0.6
```

```
Confusion Matrix:
[[1 1]
 [1 2]]
```

```
Classification Report:
              precision    recall  f1-score   support

     0       0.50      0.50      0.50         2
     1       0.67      0.67      0.67         3

   accuracy          0.60
  macro avg          0.58
weighted avg          0.60
```

```
# Predict a new sample:
# Outlook=Sunny, Temperature=Cool, Humidity=Normal, Wind=Weak
new_sample = pd.DataFrame(
    [{"Sunny", "Cool", "Normal", "Weak"}],
    columns=["Outlook", "Temperature", "Humidity", "Wind"]
)
for column in new_sample.columns:
    new_sample[column] = encoders[column].transform(new_sample[column])
prediction = model.predict(new_sample)
predicted_class = encoders["Play"].inverse_transform(prediction)[0]
print("New Sample Prediction:", predicted_class)
```

Output:

```
New Sample Prediction: Yes
```